CSCI 423
Introduction to Algorithms

DR. RAAFAT ELFOULY

Jeffrey J. McConnell

Analysis of Algorithms

Second Edition

An Active Learning Approach

Chapter 9
Parallel Algorithms

Finding the Maximum Value

- Processors compare pairs of values passing on the larger one
- Each pass needs half the processors of the previous one
- Similar to the tournament method that used one processor

Finding the Maximum Value Algorithm A

```
count = N / 2    (P number of processors)
for i = 1 to (lg count) + 1 do
   Parallel Start
      for j = 1 to count do
         Pj reads M2j-1 into X and M2j into Y
         if X > Y then
            Pj writes X into Mj
         else
            Pj writes Y into Mj
         end if
      end for
   Parallel End
   count = count / 2
end for
```

Analysis

- Each pass creates another level of the tournament tree, so there are O(lg *N*) passes
- There will be *N*/2 processors needed, giving a cost of O(*N* lg *N*)
- The sequential algorithm had a cost of only O(*N*), so there must be a better parallel version

Better Version of Finding the Maximum Value

- To get a cost of O(*N*) with a run-time of O(lg *N*), we can only use *N*/lg *N* processors

- So, we have a first step in which the processors sequentially find the maximum of lg *N* values

- We then do the original process on the results of this step

Finding the Maximum Value Algorithm B

```
Parallel Start
   for j = 1 to N/lg N do
      Pj finds the maximum of M1+(j-1)*lg N
         through Mj*lg(N) using the sequential
          algorithm
      Pj write the maximum to Mj
   end for
Parallel End
count = (N /lg N) / 2
```

Continues as in Algorithm A

Analysis

- Overall, the algorithm has a time of O(lg *N*) and a cost of O(*N*)

- So, the cost is the same, but the time is much faster than the sequential version

Parallel Searching

- A naive attempt has one processor per list element and does its work in one step
- This takes a time of 1 and a cost of O(*N*)

Parallel Searching Algorithm A

```
Parallel Start
   for j = 1 to N do
      Pj reads X from Mj and target from MN+1
      if X == target then
         write j to MN+2
      end if
   end for
Parallel End
```

Improved Parallel Searching

- If we use $p \leq N$ processors, we can have each one do a binary search on N/p elements of the list
- This gives a time of $O(\lg (N/p))$ and a cost of $O(p \lg (N/p))$
- If $p = \lg N$,
  - we get a run time of $O(\lg N/p) = (\log N - \log p) = \text{Log N - Log Log N}$
  - and a cost of $O(p \lg (N/p)) = O(\lg^2 N)$
- Remember:
  - $\log_b(mn) = \log_b(m) + \log_b(n)$
  - $\log_b(m/n) = \log_b(m) - \log_b(n)$
  - $\log_b(m^n) = n \cdot \log_b(m)$

Parallel Searching Algorithm B

Using $p \leq N$ processors

Parallel Start

    for j = 1 to p do

        P_j performs a binary search on

                $M_{(j-1)*(N/p)+1}$ through $M_{j*(N/p)}$ writing the

                location where X is found to M_{N+2}

    end for

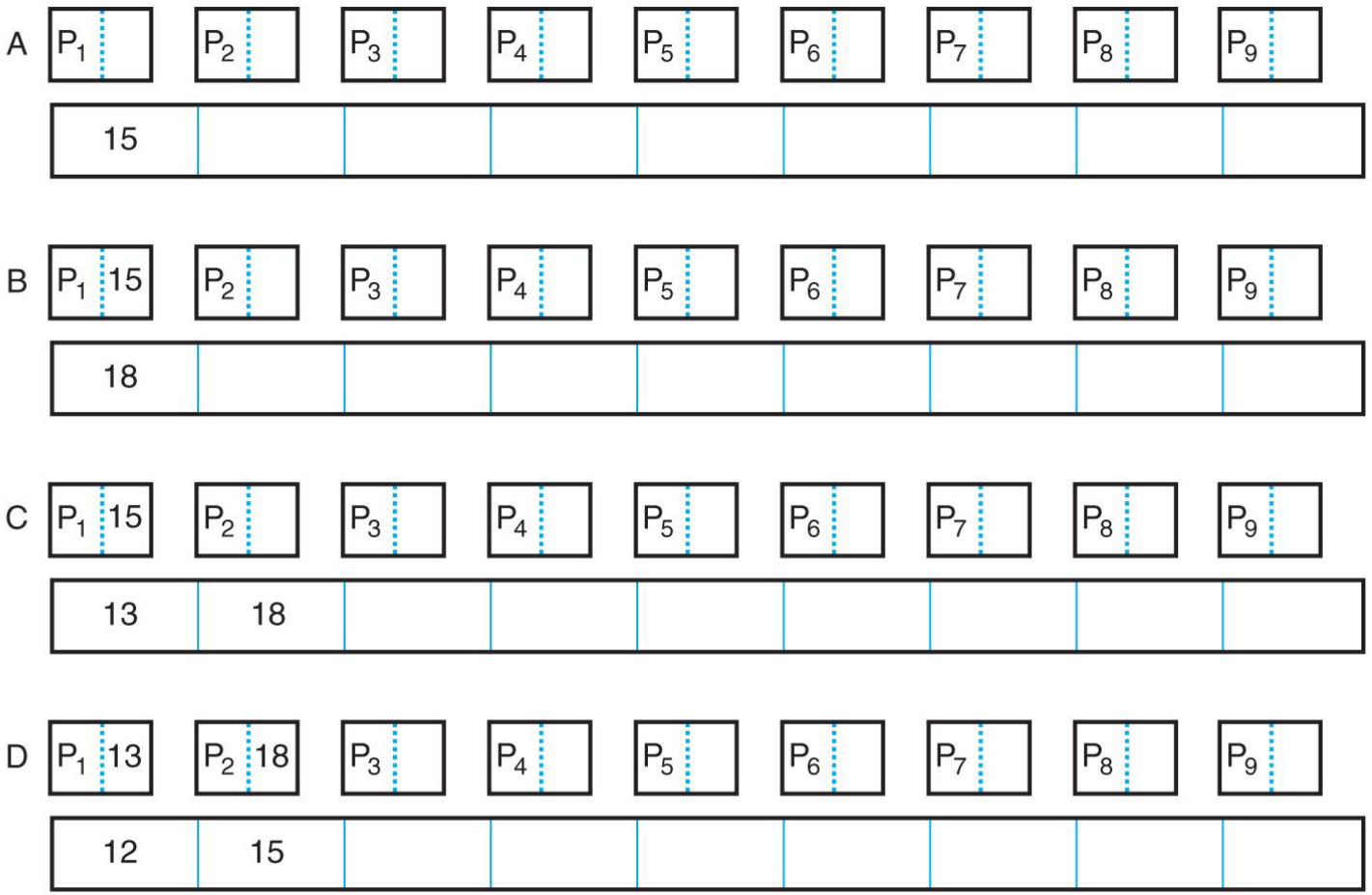
Parallel End

Linear Network Sorting

- Uses *N* processors
- On each pass, the processor compares its current value with a new value, and passes the larger of the two onto the next processor
- The smallest value will wind up with the first processor and the largest value will wind up with the last processor

Linear Sort Example

FIGURE 9.4A
A linear parallel network sort



Linear Sort Example

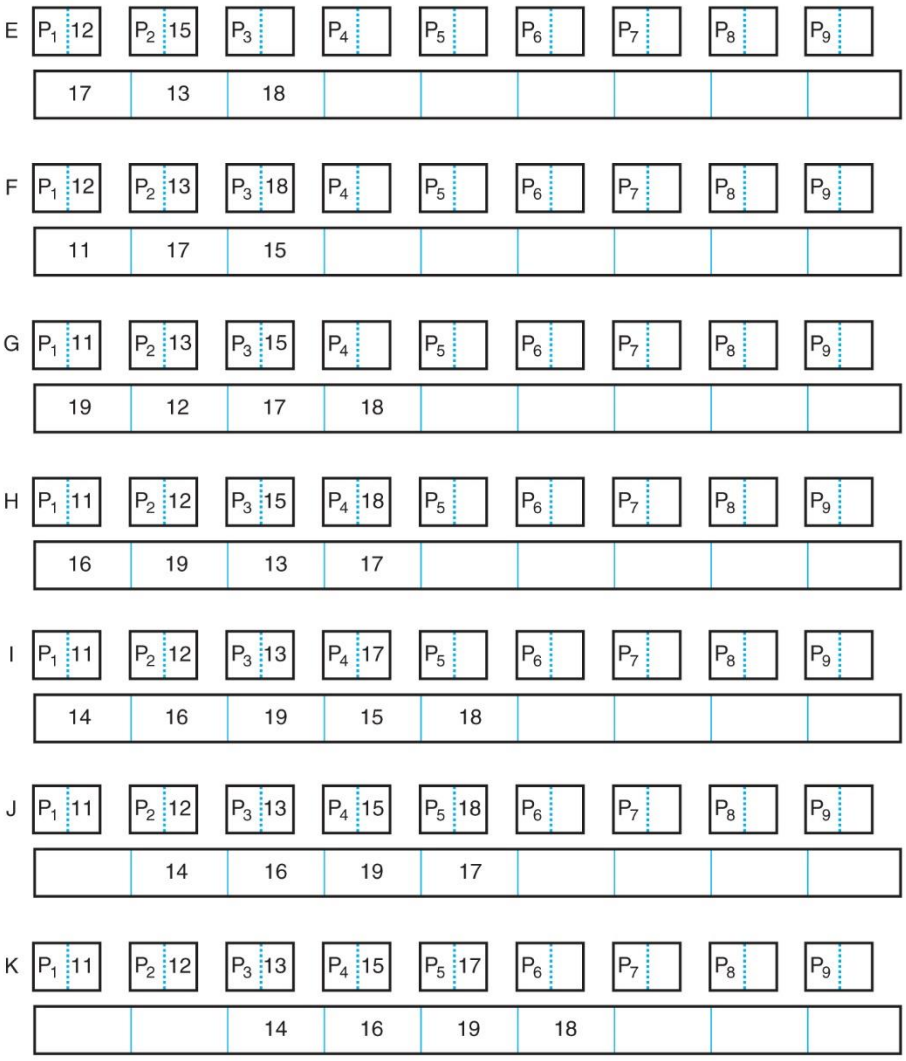


FIGURE 9.4B
A linear parallel network sort

Linear Sort Example

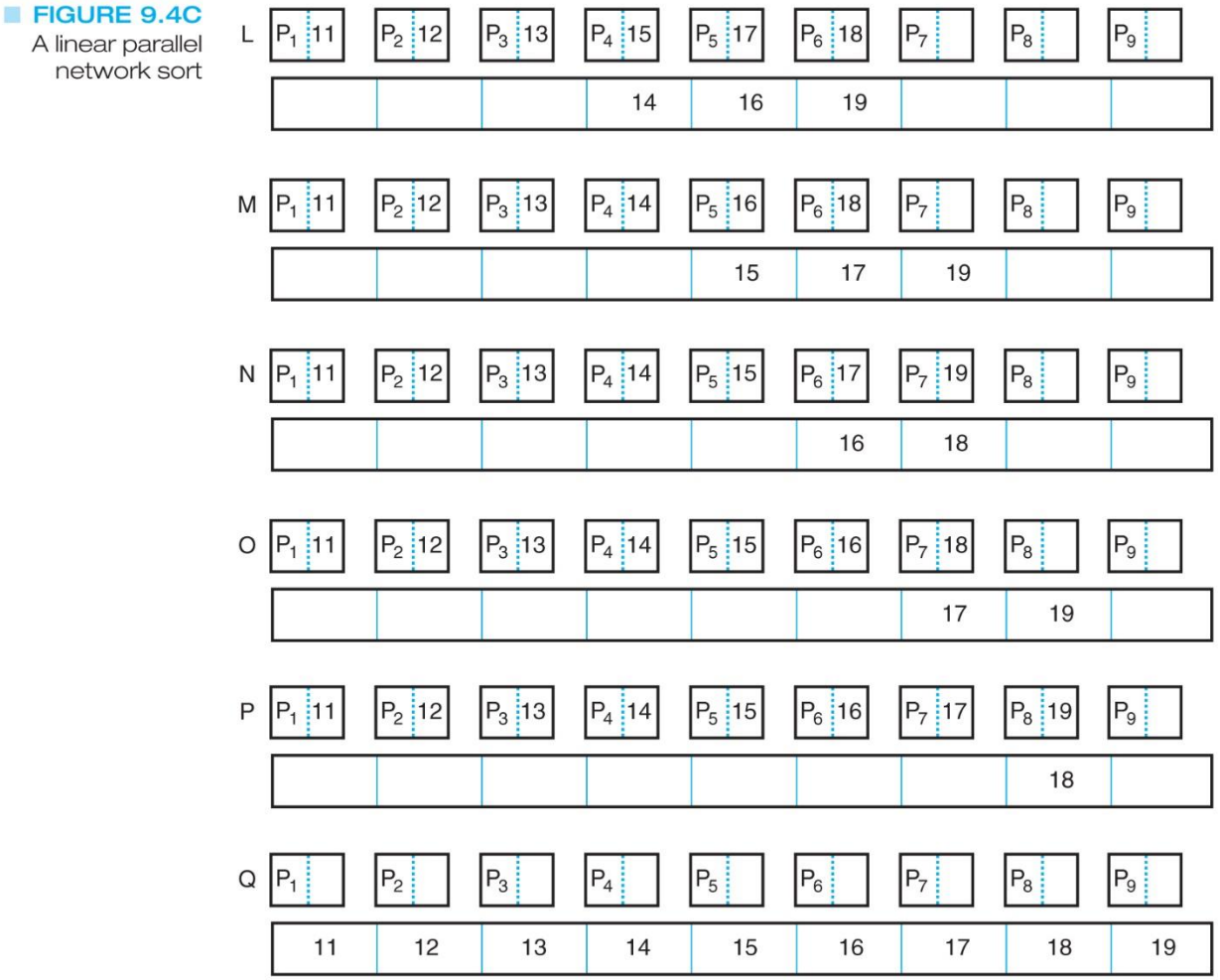


FIGURE 9.4C
A linear parallel network sort

Try to develop Algorithm

Linear Network Sort Part A

```
for j = 1 to N-1 do
   put next value into M1
   Parallel Start
      Pj reads Mj into Current
      for k = 1 to j-1 do
         Pk reads Mk into New
         if Current > New then
            Pk writes Current to Mk+1
            Current = New
         Else
            Pk writes New to Mk+1
         end if
      end for k
   Parallel End
```

Linear Network Sort Part B

```
    for j = 1 to N-1 do
     put next value into M1
     Parallel Start
        for k = 1 to j-1 do
            Pk reads Mk into New
            if Current > New then
                Pk writes Current to Mk+1
                Current = New
            Else
                Pk writes New to Mk+1
            end if
        end for k
     Parallel End
end for j
Parallel Start
    for j = 1 to N-1 do
        Pj writes Current to Mj
    end for j
Parallel End
```

Analysis

- This process uses *N* processors and has a run time of O(*N*) and a cost of O(N^2)

- This is the same cost as the slower sorts

Recall: Matrix Multiplication

- Two matrices can be multiplied if the number of columns in the first is the same as the number of rows in the second

Recall: Matrix Multiplication

- Two matrices can be multiplied if the number of columns in the first is the same as the number of rows in the second

- Each row of the first matrix is multiplied by each column of the second matrix

Recall: Matrix Multiplication

- Two matrices can be multiplied if the number of columns in the first is the same as the number of rows in the second
- Each row of the first matrix is multiplied by each column of the second matrix
- The value at location *i*, *j* of the result is from the addition of the products when row *i* of the first matrix is multiplied by row *j* of the second matrix

Matrix Multiplication

- An example of this process is:

$$\begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} a1+b2+c3 & a4+b5+c6 \\ d1+e2+f3 & d4+e5+f6 \end{bmatrix}$$

Matrix Multiplication Algorithm

```
for i = 1 to a do
   for j = 1 to c do
      R[i,j] = G[i,1] * H[1,j]
      for k = 2 to b
         R[i,j] = R[i,j] + G[i,k] * H[k,j]
      end for k
   end for j
end for i
```

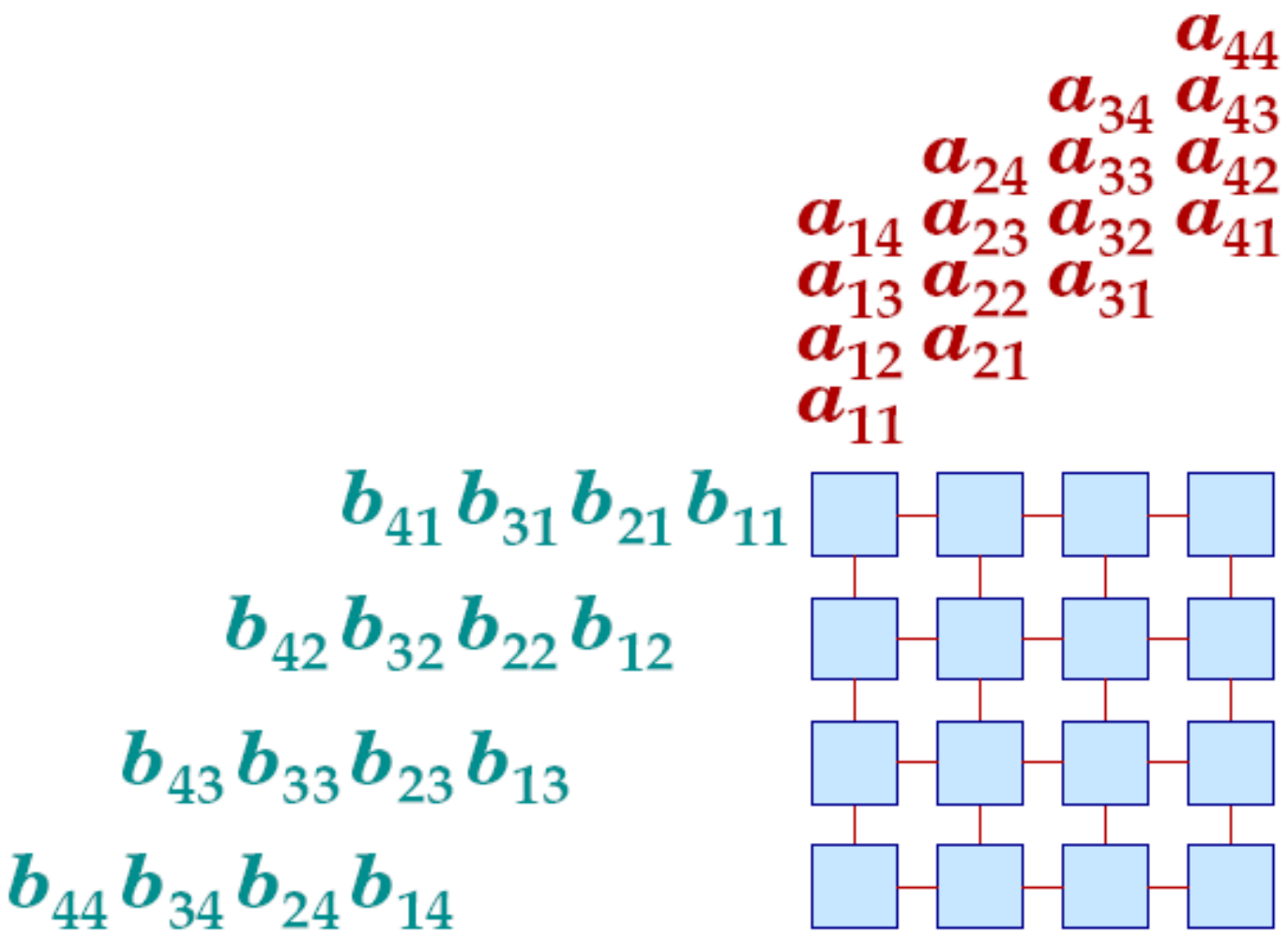
Analysis

- The "for *k*" loop runs *b* – 1 times and does one multiplication and addition each time
- The "for *j*" loop runs *c* times and does *b* multiplications and *b* – 1 additions each time
- The "for *i*" loop runs a times and does *b***c* multiplications and (*b* – 1)**c* additions each time
- Therefore, the algorithm does *a***b***c* multiplications $O(N^3)$

Matrix Multiplication on a Parallel Mesh

- The mesh has one row of processors for each row of the first matrix and one column of processors for each column of the second matrix

Matrix-Matrix Multiplication



Matrix Multiplication on a Parallel Mesh

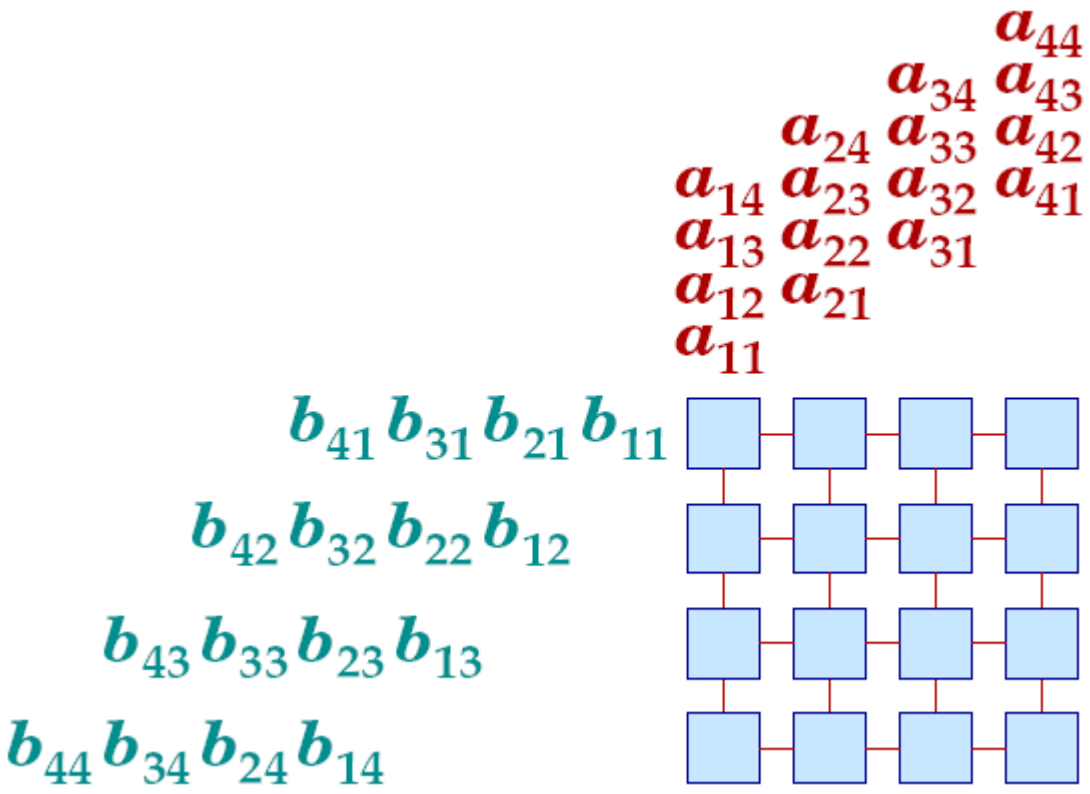
- The mesh has one row of processors for each row of the first matrix and one column of processors for each column of the second matrix

- The values are passed through the mesh in a staggered fashion so that pairs of values that need to be multiplied arrive at the same time

Matrix Multiplication on a Parallel Mesh

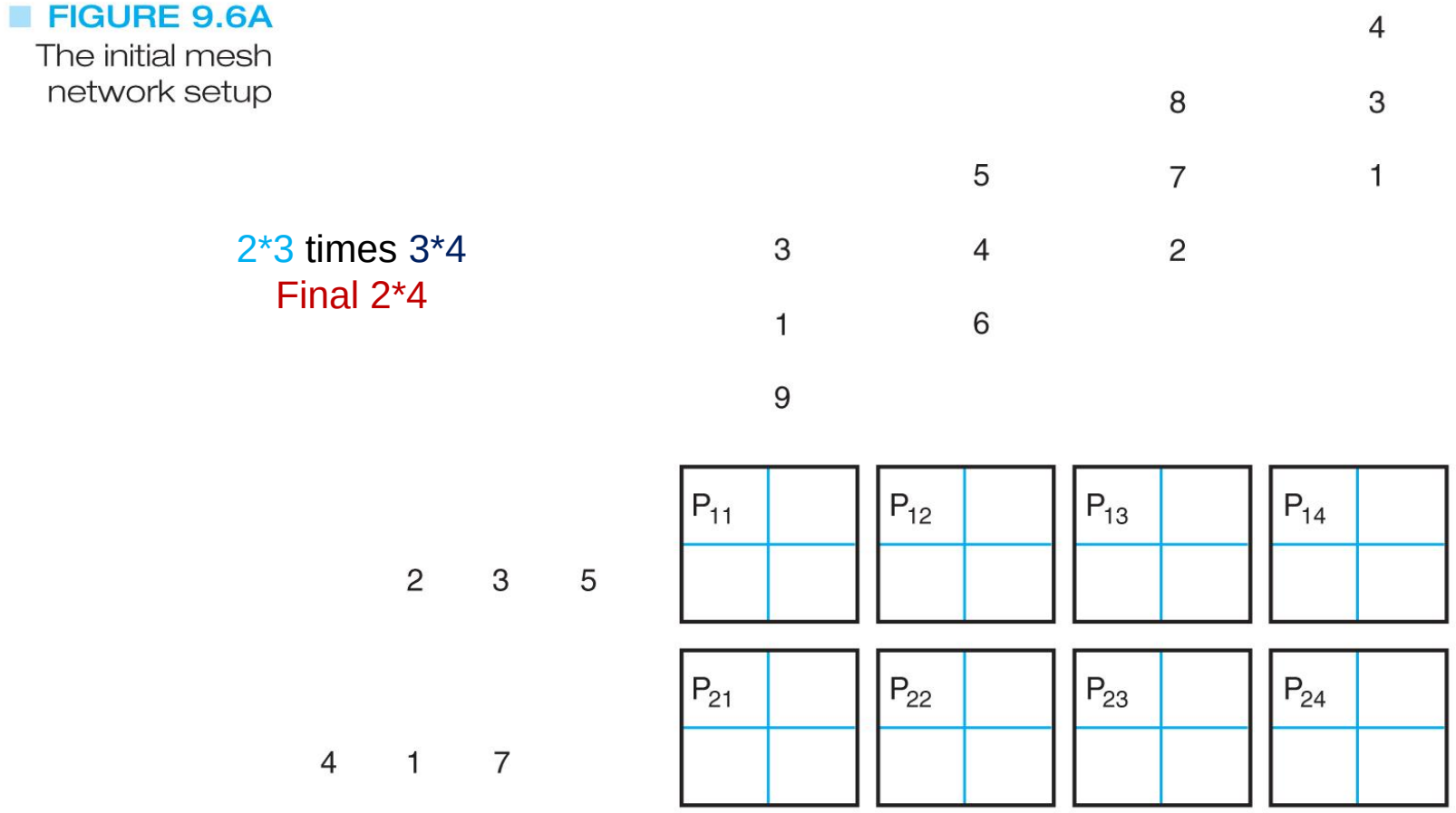
- The mesh has one row of processors for each row of the first matrix and one column of processors for each column of the second matrix
- The values are passed through the mesh in a staggered fashion so that pairs of values that need to be multiplied arrive at the same time
- The processors keep a running total of the values they have multiplied and have the result at the end of the task

Matrix-Matrix Multiplication



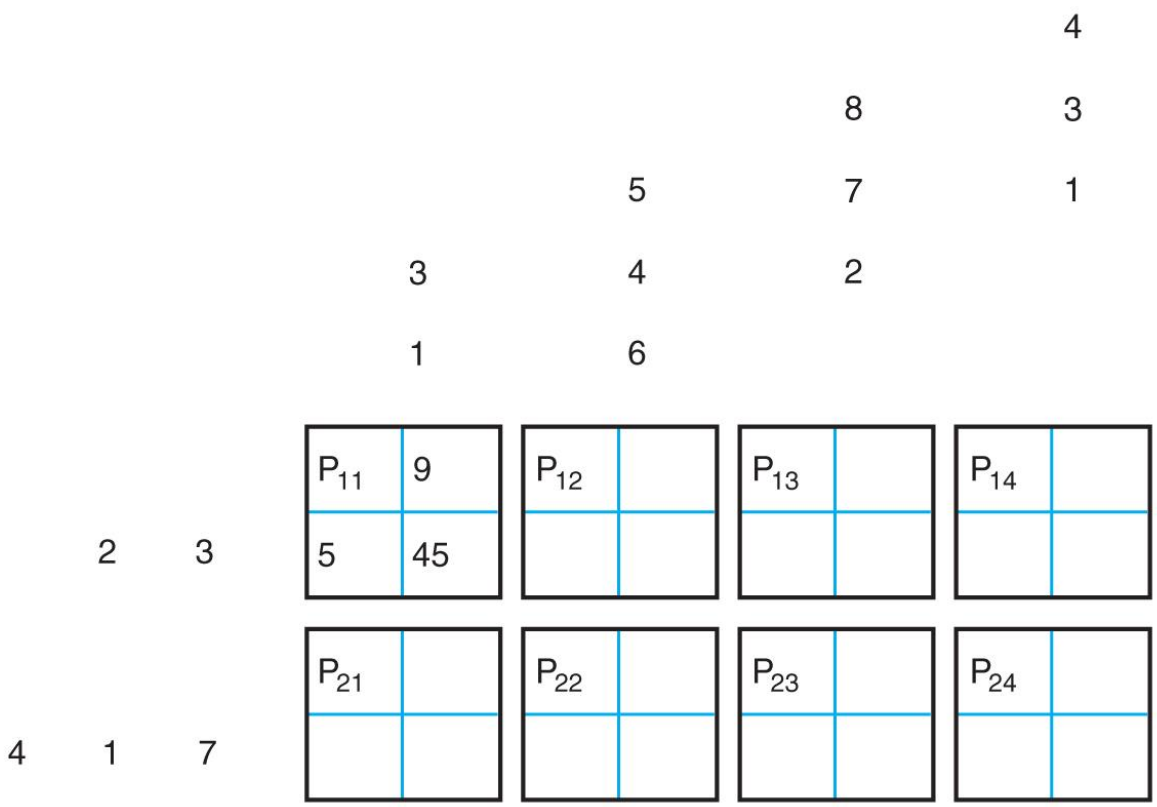
Matrix Multiplication on a Parallel Mesh Example

FIGURE 9.6A
The initial mesh network setup



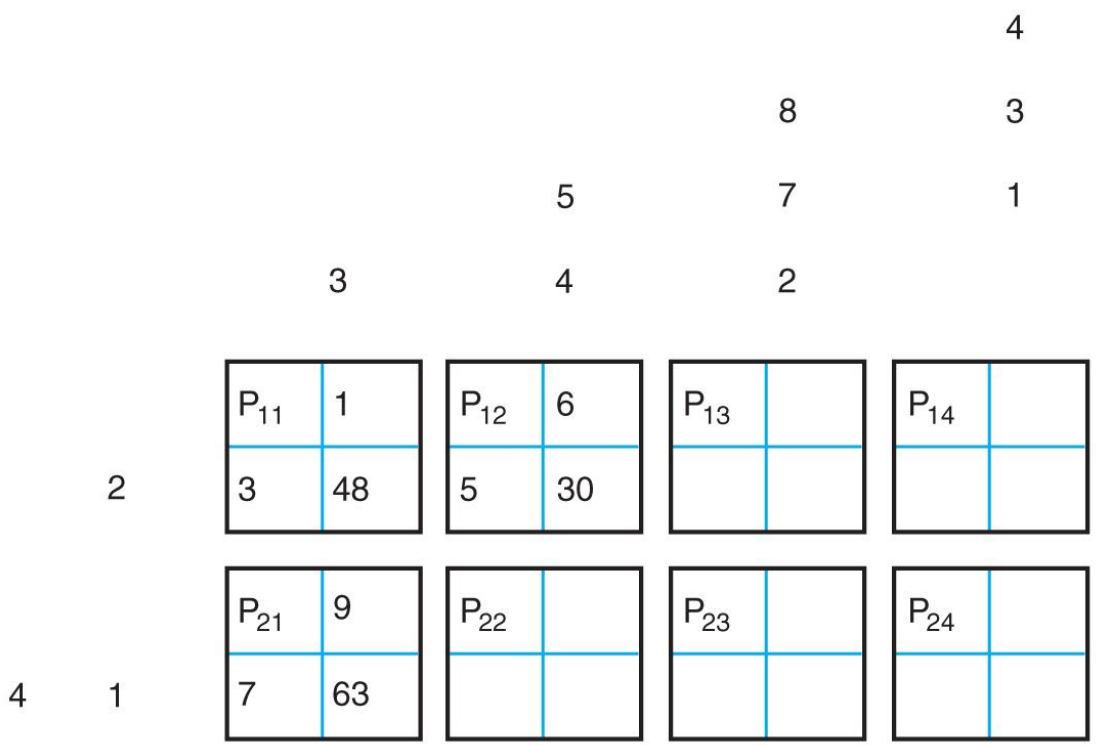
Matrix Multiplication on a Parallel Mesh Example (continued)

FIGURE 9.6B
The first two values are multiplied by P_{11} and stored in its register



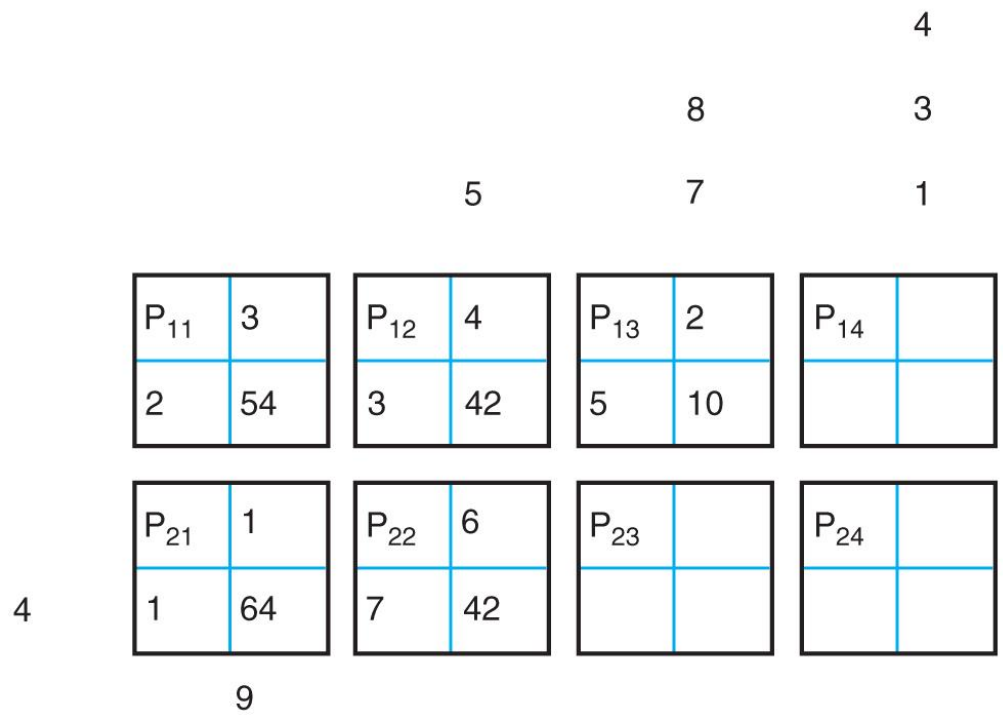
Matrix Multiplication on a Parallel Mesh Example (continued)

FIGURE 9.6C
P_{11} multiplies the next two values and adds the result to its register while P_{12} and P_{21} multiply their first two numbers



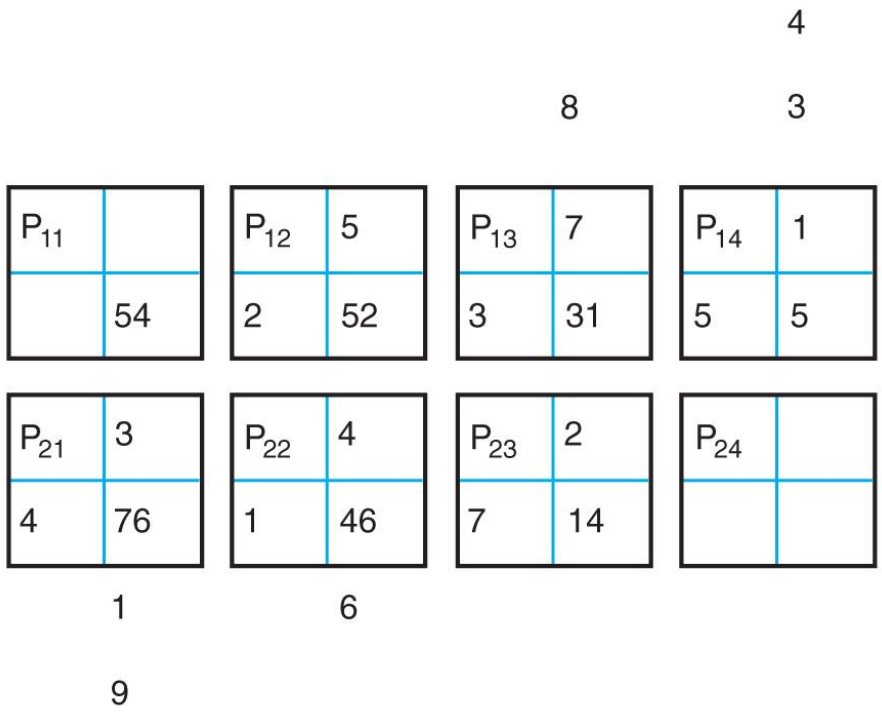
Matrix Multiplication on a Parallel Mesh Example (continued)

FIGURE 9.6D
P_{11}, P_{12}, and P_{21} handle their next two numbers, while P_{13} and P_{22} join the process



Matrix Multiplication on a Parallel Mesh Example (continued)

FIGURE 9.6E
P_{11} is now finished and the result for row 1, column 1 is in its register; P_{12}, P_{21}, P_{13}, and P_{22} work on their next two values; and P_{14} and P_{23} get started



Matrix Multiplication on a Parallel Mesh Example (continued)

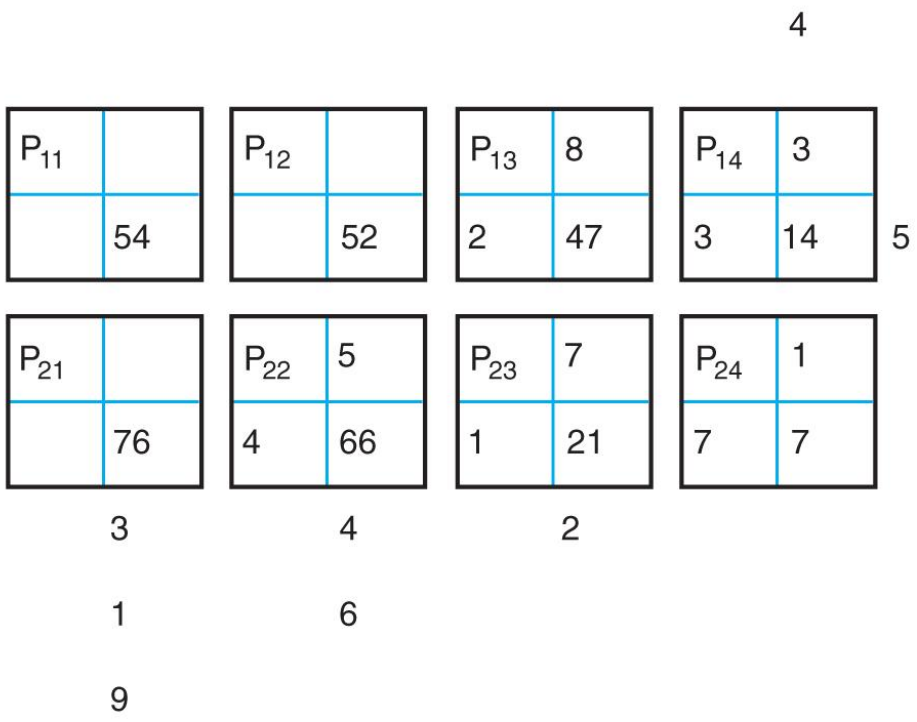


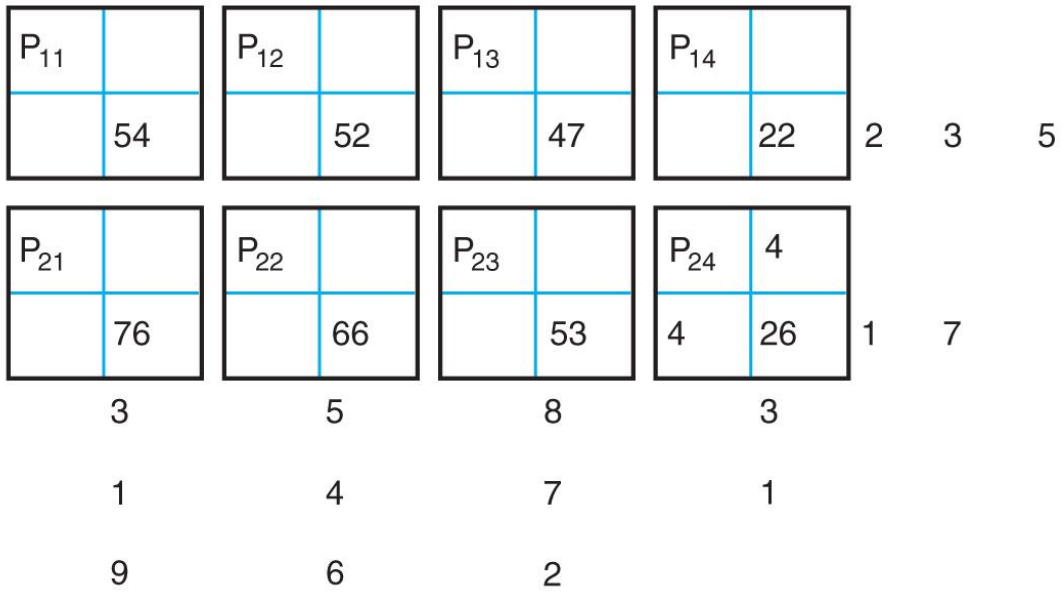
FIGURE 9.6F
P_{12} and P_{21} are now done; P_{24} gets its first two numbers; and P_{13}, P_{22}, P_{14}, and P_{23} work on their next values

Matrix Multiplication on a Parallel Mesh Example (continued)

FIGURE 9.6G
P_{13} and P_{22} are now done, while P_{14}, P_{23}, and P_{24} work on their next values

Matrix Multiplication on a Parallel Mesh Example (continued)

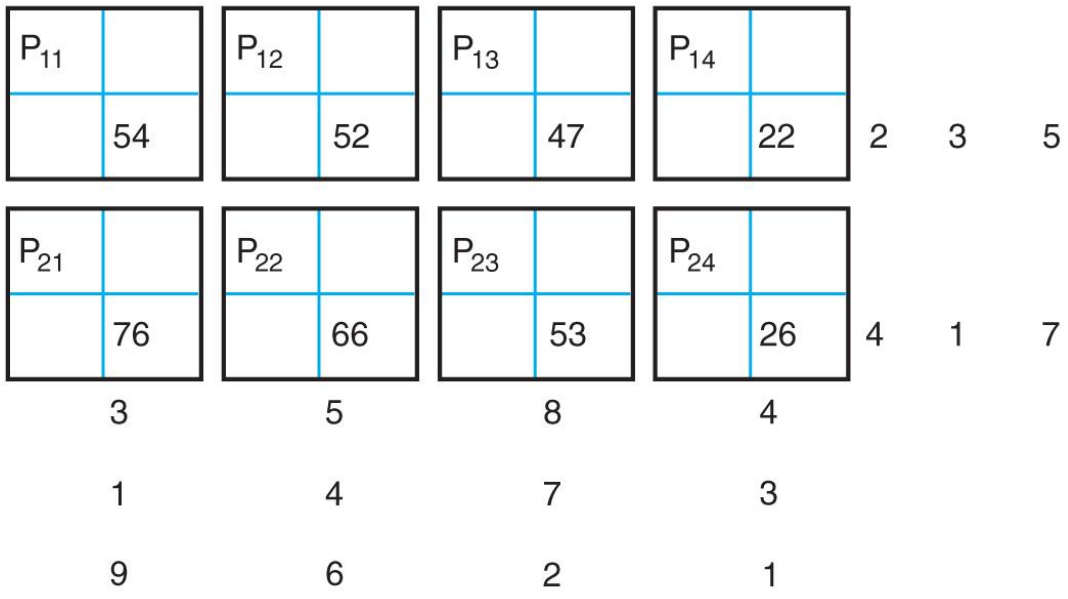
FIGURE 9.6H
P_{14} and P_{23} finish, while P_{24} works on its last two values



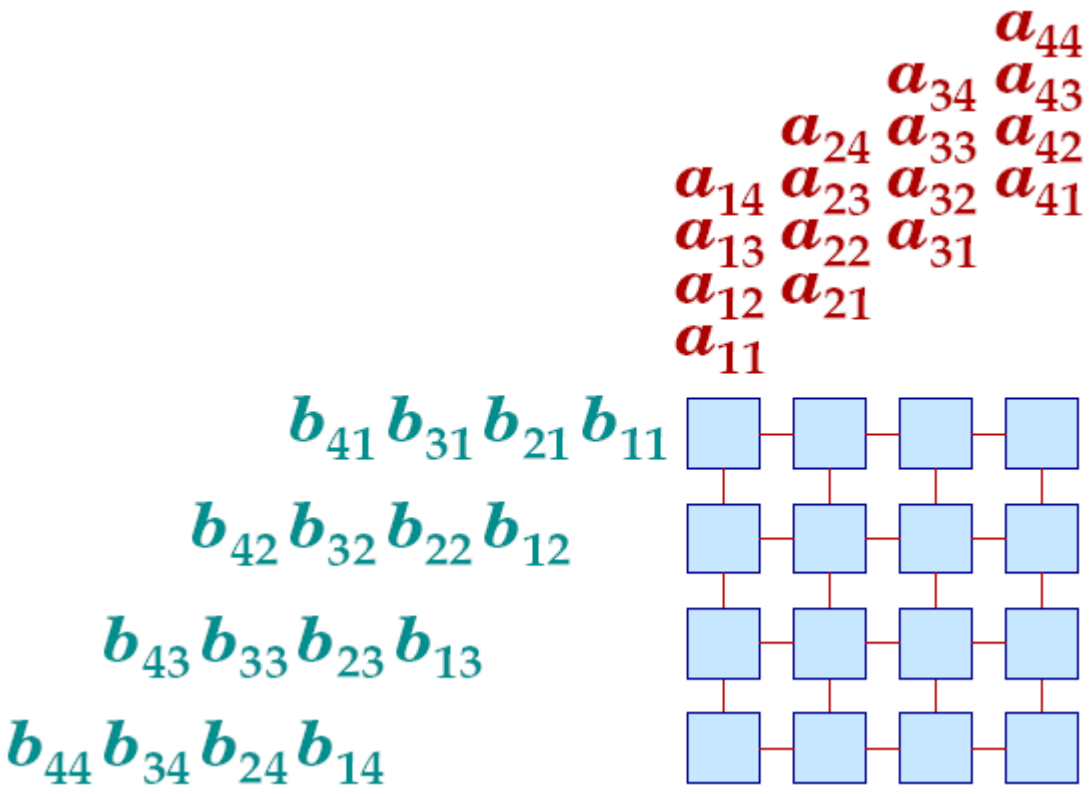
Matrix Multiplication on a Parallel Mesh Example (continued)

FIGURE 9.6I
The multiplication is done and the processors hold the result of

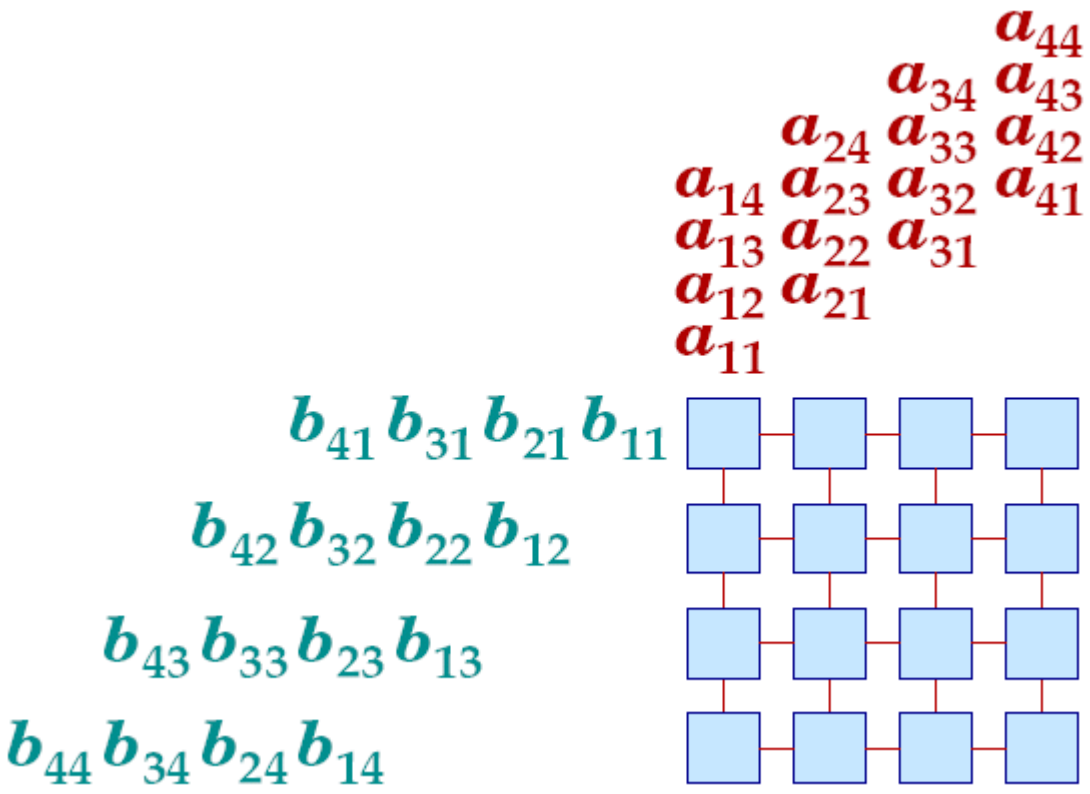
$$\begin{bmatrix} 54 & 52 & 47 & 22 \\ 76 & 66 & 53 & 26 \end{bmatrix}$$



Matrix-Matrix Multiplication



Matrix-Matrix Multiplication



Number of time steps = 3N – 2

Analysis

- The run time of this algorithm is $O(N)$ and the cost is $O(N^3)$, where N is the maximum of the matrix sizes

Matrix Multiplication in a CRCW PRAM Model Algorithm

```
Parallel Start
   for x = 1 to I do
      for y = 1 to J do
         for z = 1 to K do
            P_xyz calculates A_xy * B_yz and
                  stores it in M_xz
         end for
      end for
   end for
Parallel End
```

Analysis

- Each processor is responsible for one of the $O(N^3)$ multiplications
- The run time is 1 and the cost is $O(N^3)$
- CW M_{xz} will add partial results automatically using "Combine"

Dynamic Programming

- Problems for which the most efficient solution is dependent on choices made not predetermined choices

- In other words, the algorithm must be able to change its behavior based on how things have gone earlier in the processing

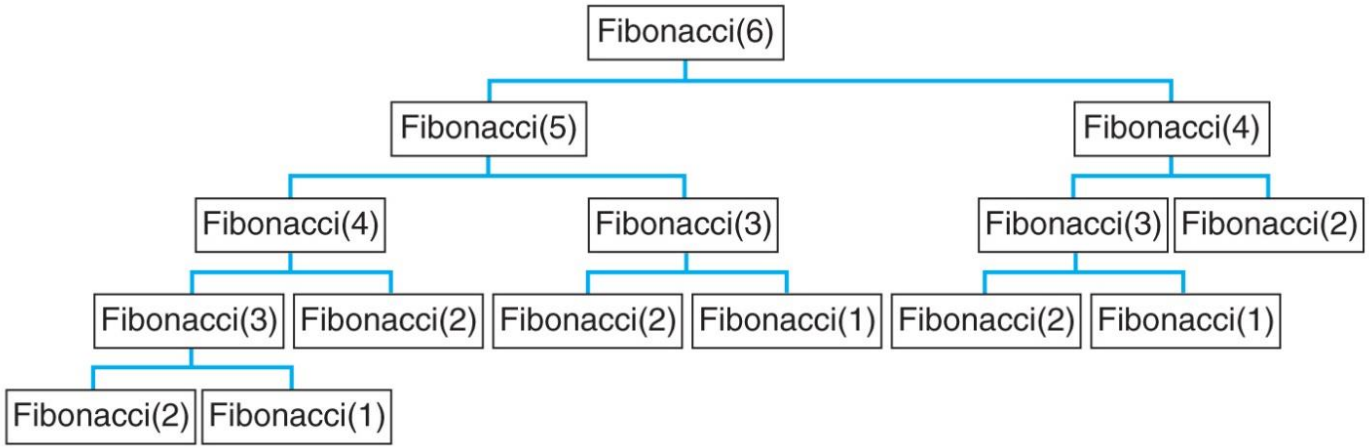
Array-Based Methods

- Use arrays to keep track of information that can be used instead of being calculated

Fibonacci Numbers

- The classic recursive Fibonacci numbers algorithm will recalculate values multiple times

FIGURE 11.8
The calling sequence for Fibonacci(6)



Fibonacci Numbers

- The classic recursive Fibonacci numbers algorithm will recalculate values multiple times

- An array-based alternative would save the smaller Fibonacci numbers as they are calculated and use those values instead of recalculating

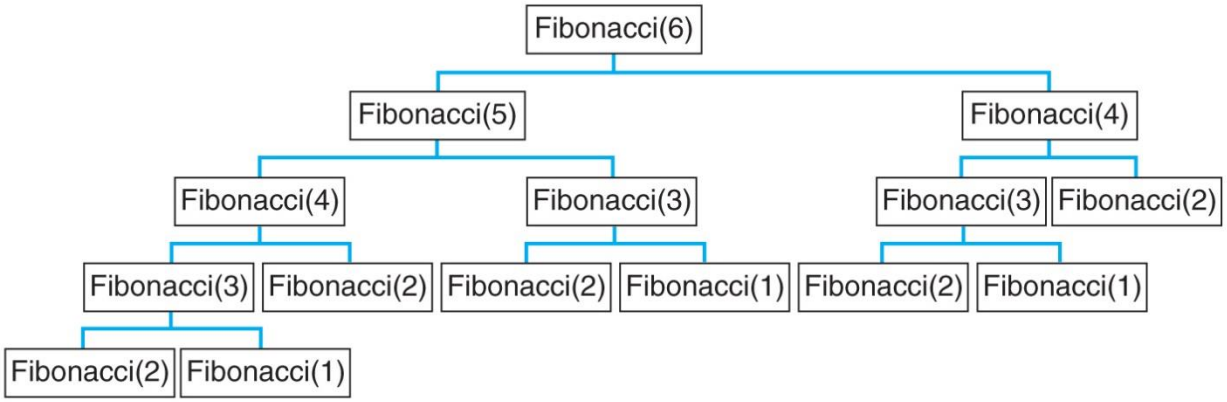


FIGURE 11.8 The calling sequence for Fibonacci(6)

Dynamic Matrix Multiplication

- If we have a series of matrices of different sizes that we need to multiply, the order we do it can have a big impact on the number of operations

Dynamic Matrix Multiplication

- If we have a series of matrices of different sizes that we need to multiply, the order we do it can have a big impact on the number of operations

- For example, if we need to multiply four matrices of sizes 20X5, 5X35, 35X4, and 4X25, depending on the order this can take between 3100 and 24,500 multiplications

Dynamic Matrix Multiplication Process

- We look at the ways to pair the matrices and then combine these into groups of three, and then four

- We keep track of the efficiency of these partial results and build on that

Dynamic Matrix Multiplication Example

FIGURE 11.9
The amount of work needed to multiply four matrices in different orders

Multiplication Order	Resulting Matrix Size	Total Cost in Multiplications
$M_1 * M_2$	20 × 35	3500
$M_2 * M_3$	5 × 4	700
$M_3 * M_4$	35 × 25	3500
$(M_1 * M_2) * M_3$	20 × 4	3500 + 2800 = 6300
$M_1 * (M_2 * M_3)$	20 × 4	700 + 400 = 1100
$(M_2 * M_3) * M_4$	5 × 25	700 + 500 = 1200
$M_2 * (M_3 * M_4)$	5 × 25	3500 + 4375 = 7875
$((M_1 * M_2) * M_3) * M_4$	20 × 25	6300 + 2000 = 8300
$(M_1 * (M_2 * M_3)) * M_4$	20 × 25	1100 + 2000 = 3100
$M_1 * ((M_2 * M_3) * M_4)$	20 × 25	1200 + 2500 = 3700
$M_1 * (M_2 * (M_3 * M_4))$	20 × 25	7875 + 2500 = 10,375
$(M_1 * M_2) * (M_3 * M_4)$	20 × 25	3500 + 3500 + 17,500 = 24,500

The Class P

- The algorithms that were considered in chapters 2 though 9 are considered polynomial time algorithms because they all have run times or costs that are polynomial functions

- Together these algorithms are said to be in the class P

What is the Class NP?

- Problems in class NP are those for which the only known polynomial time algorithm must have a nondeterministic step

- Typically, the solution has a first step that nondeterministically chooses a potential solution, and then a second step checks to see if that solution is correct

Class P $\subseteq$ Class NP

- The class P is a subset of the class NP, because we could construct an algorithm that solved the class P problems with the same two step process that is used for class NP problems

- The difference is that we have polynomial time solutions for the problems in class P, but we do not know of one for class NP problems